

Министерство науки и высшего образования РФ
Федеральное государственное автономное образовательное учреждение
высшего образования “Омский государственный технический университет”
Кафедра “Автоматизированные системы обработки информации и управления”

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ № 5
по дисциплине «Инженерия тестирования»
студентов Рябченко И.С., Бабиковой С.Д. ПИН-252т

Пояснительная записка
Шифр работы От-2068998-43-ПИН-252т ЛР
Специальность 09.03.03 Программная инженерия

Преподаватель

В.А. Никонов

Студенты

И.С. Рябченко

С.Д. Бабикова

Омск 2026

Задание лабораторной работы №5

Создание модульных тестов на основании тестовых сценариев.

На основании сформированных тестовых сценариев реализовать модульные тесты. Так же реализовать тест-кейсы, соответствующие проверке интерфейсной части ПО.

Введение

Данная лабораторная работа посвящена реализации модульных тестов для калькулятора комплексных чисел на основании тестовых сценариев, разработанных в предыдущей лабораторной работе.

Модульные тесты написаны на платформе *MSTest* и проверяют как класс *ComplexNum* (арифметика, сравнение, модуль, форматирование вывода), так и интерфейсную часть — форму *MainForm* (валидация полей ввода, выбор операции, отображение результата). Каждый тест соответствует одному из тестовых сценариев и проверяет одно ожидаемое поведение. Все тесты прошли успешно (28 из 28).

Реализация модульных тестов

Для тестирования класса *ComplexNum* используется атрибут *[TestClass]* и метод-тест с атрибутом *[TestMethod]*. Каждый тест создаёт входные данные, вызывает проверяемый метод и сравнивает результат с ожидаемым через *Assert.AreEqual*. Для проверки исключения деления на ноль применяется *Assert.ThrowsException<DivideByZeroException>*.

```
using Microsoft.VisualStudio.TestTools.UnitTesting;
using ComplexCalculatorContracts;

namespace ClassLibraryForCalc.Tests
{
    [TestClass]
    Ссылка: 0
    public class ComplexNumTests
    {
        [TestMethod]
        Ссылка: 0
        public void Test01_CreateComplexNumber()
        {
            ComplexNum z = new ComplexNum(3, 4);
            Assert.AreEqual(3, z.RPart);
            Assert.AreEqual(4, z.ImPart);
        }

        [TestMethod]
        Ссылка: 0
        public void Test02_AddTwoNumbers()
        {
            ComplexNum a = new ComplexNum(2, 3);
            ComplexNum b = new ComplexNum(1, 4);
            IComplexNumber r = a.Plus(b);
            Assert.AreEqual(3, r.RPart);
            Assert.AreEqual(7, r.ImPart);
        }

        [TestMethod]
        Ссылка: 0
        public void Test03_AddZero()
        {
            ComplexNum a = new ComplexNum(5, 2);
            ComplexNum b = new ComplexNum(0, 0);
            IComplexNumber r = a.Plus(b);
            Assert.AreEqual(5, r.RPart);
            Assert.AreEqual(2, r.ImPart);
        }

        [TestMethod]
        Ссылка: 0
        public void Test04_SubtractTwoNumbers()
        {
            ComplexNum a = new ComplexNum(5, 7);
            ComplexNum b = new ComplexNum(2, 3);
            IComplexNumber r = a.Minus(b);
            Assert.AreEqual(3, r.RPart);
            Assert.AreEqual(4, r.ImPart);
        }
    }
}
```

Рисунок-1 Тесты создания объекта и сложения/вычитания

Начало класса *ComplexNumTests*: тесты создания объекта *ComplexNum* и проверки операций сложения (с ненулевым и нулевым слагаемым) и вычитания двух комплексных чисел.

```
public void Test05_SubtractEqual()
{
    ComplexNum a = new ComplexNum(3, 5);
    ComplexNum b = new ComplexNum(3, 5);
    IComplexNumber r = a.Minus(b);
    Assert.AreEqual(0, r.RPart);
    Assert.AreEqual(0, r.ImPart);
}

[TestMethod]
Ссылка: 0
public void Test06_MultiplyTwoNumbers()
{
    ComplexNum a = new ComplexNum(2, 3);
    ComplexNum b = new ComplexNum(1, 4);
    IComplexNumber r = a.Mnoj(b);
    Assert.AreEqual(-10, r.RPart);
    Assert.AreEqual(11, r.ImPart);
}

[TestMethod]
Ссылка: 0
public void Test07_MultiplyByZero()
{
    ComplexNum a = new ComplexNum(7, 9);
    ComplexNum b = new ComplexNum(0, 0);
    IComplexNumber r = a.Mnoj(b);
    Assert.AreEqual(0, r.RPart);
    Assert.AreEqual(0, r.ImPart);
}

[TestMethod]
Ссылка: 0
public void Test08_DivideTwoNumbers()
{
    ComplexNum a = new ComplexNum(4, 2);
    ComplexNum b = new ComplexNum(1, 1);
    IComplexNumber r = a.Delen(b);
    Assert.AreEqual(3, r.RPart);
    Assert.AreEqual(-1, r.ImPart);
}

[TestMethod]
Ссылка: 0
public void Test09_DivideByZero()
{
    ComplexNum a = new ComplexNum(5, 3);
    ComplexNum b = new ComplexNum(0, 0);
    Assert.ThrowsException<DivideByZeroException>(() => a.Delen(b));
}

[TestMethod]
Ссылка: 0
public void Test10_ComputeModulus()
{
    ComplexNum z = new ComplexNum(3, 4);
    Assert.AreEqual(5, z.Modul);
}
```

Рисунок-2 Тесты умножения, деления и модуля

Тесты арифметических операций класса *ComplexNum*: вычитание равных чисел, умножение (в том числе на нуль), деление, а также проверка генерации исключения *DivideByZeroException* и вычисления модуля числа.

```

[TestMethod]
Ссылка: 0
public void Test11_ZeroModulus()
{
    ComplexNum z = new ComplexNum(0, 0);
    Assert.AreEqual(0, z.Modul);
}

[TestMethod]
Ссылка: 0
public void Test12_EqualNumbers()
{
    ComplexNum a = new ComplexNum(2, 3);
    ComplexNum b = new ComplexNum(2, 3);
    Assert.IsTrue(a.Equals(b));
}

[TestMethod]
Ссылка: 0
public void Test13_NotEqualNumbers()
{
    ComplexNum a = new ComplexNum(2, 3);
    ComplexNum b = new ComplexNum(5, 3);
    Assert.IsFalse(a.Equals(b));
}

[TestMethod]
Ссылка: 0
public void Test14_SmallerByModulus()
{
    ComplexNum a = new ComplexNum(1, 1);
    ComplexNum b = new ComplexNum(3, 4);
    Assert.IsTrue(a.Smaller(b));
}

[TestMethod]
Ссылка: 0
public void Test15_BiggerByModulus()
{
    ComplexNum a = new ComplexNum(6, 8);
    ComplexNum b = new ComplexNum(3, 4);
    Assert.IsTrue(a.Bigger(b));
}

[TestMethod]
Ссылка: 0
public void Test16_EqualModulus()
{
    ComplexNum a = new ComplexNum(3, 4);
    ComplexNum b = new ComplexNum(-4, -3);
    Assert.IsTrue(a.SmallerOrEqual(b));
}

[TestMethod]
Ссылка: 0
public void Test19_FractionalNumbers()
{
    ComplexNum a = new ComplexNum(2.5, -1.75);
    ComplexNum b = new ComplexNum(1.5, 0.25);
    IComplexNumber r = a.Plus(b);
    Assert.AreEqual(4.0, r.RPart);
    Assert.AreEqual(-1.5, r.ImPart);
}

```

Рисунок-3 Тесты сравнения комплексных чисел

Тесты сравнения и вспомогательных операций: модуль нулевого числа, проверка равенства и неравенства, сравнение по модулю, а также сложение дробных комплексных чисел.

```

[TestMethod]
Ссылка: 0
public void Test23_SubtractZero()
{
    ComplexNum a = new ComplexNum(5, 2);
    ComplexNum b = new ComplexNum(0, 0);
    IComplexNumber r = a.Minus(b);
    Assert.AreEqual(5, r.RPart);
    Assert.AreEqual(2, r.ImPart);
}

[TestMethod]
Ссылка: 0
public void Test24_DivideZeroByNumber()
{
    ComplexNum a = new ComplexNum(0, 0);
    ComplexNum b = new ComplexNum(2, 3);
    IComplexNumber r = a.Delen(b);
    Assert.AreEqual(0, r.RPart);
    Assert.AreEqual(0, r.ImPart);
}

[TestMethod]
Ссылка: 0
public void Test25_DivideEqualNumbers()
{
    ComplexNum a = new ComplexNum(3, 5);
    ComplexNum b = new ComplexNum(3, 5);
    IComplexNumber r = a.Delen(b);
    Assert.AreEqual(1, r.RPart);
    Assert.AreEqual(0, r.ImPart);
}

[TestMethod]
Ссылка: 0
public void Test26_BiggerOrEqualModulus()
{
    ComplexNum a = new ComplexNum(3, 4);
    ComplexNum b = new ComplexNum(-4, -3);
    Assert.IsTrue(a.BiggerOrEqual(b));
}

[TestMethod]
public void Test27_ToStringPositiveImaginary()
{
    ComplexNum z = new ComplexNum(2, 3);
    Assert.AreEqual("2 + 3i", z.ToString());
}

[TestMethod]
public void Test28_ToStringNegativeImaginary()
{
    ComplexNum z = new ComplexNum(2, -3);
    Assert.AreEqual("2 - 3i", z.ToString());
}
}

```

Рисунок-4 Тесты граничных случаев и форматирования

Дополнительные тесты арифметики и форматирования вывода: вычитание нуля, деление нуля на число, деление равных чисел, сравнение по модулю, а также метод *ToString()* для положительной и отрицательной мнимой части.

Тесты интерфейсной части формы *MainForm* получают доступ к её элементам управления через *Controls.Find* и эмулируют нажатие кнопок прямым вызовом обработчиков *OnClick* и *CalcClick*. Это позволяет проверить, что валидация полей, выбор операции и вывод результата работают согласно спецификации без запуска главного цикла приложения.

```
using Microsoft.VisualStudio.TestTools.UnitTesting;
using System.Windows.Forms;
using ComplexCalculatorContracts;

namespace ClassLibraryForCalc.Tests
{
    [TestClass]
    public class MainFormTests
    {
        TextBox FindTextBox(MainForm form, string name)
        {
            return (TextBox)form.Controls.Find(name, true)[0];
        }

        Button FindButton(MainForm form, string name)
        {
            return (Button)form.Controls.Find(name, true)[0];
        }

        Label FindLabel(MainForm form, string name)
        {
            return (Label)form.Controls.Find(name, true)[0];
        }

        [TestMethod]
        public void Test17_LettersInsteadOfNumber()
        {
            MainForm form = new MainForm();
            FindTextBox(form, "relBox").Text = "abc";
            FindTextBox(form, "imlBox").Text = "2";

            ICalculatorUI ui = form;
            try
            {
                ui.ReadComplexNumber("first");
                Assert.Fail();
            }
            catch (ArgumentException ex)
            {
                Assert.AreEqual("некорректный ввод", ex.Message);
            }
        }

        [TestMethod]
        public void Test18_EmptyField()
        {
            MainForm form = new MainForm();
            FindTextBox(form, "relBox").Text = "";
            FindTextBox(form, "imlBox").Text = "3";

            ICalculatorUI ui = form;
            try
            {
                ui.ReadComplexNumber("first");
                Assert.Fail();
            }
            catch (ArgumentException ex)
            {
                Assert.AreEqual("поле не должно быть пустым", ex.Message);
            }
        }
    }
}
```

Рисунок-5 Тесты валидации ввода в *MainForm*

Начало класса *MainFormTests*: вспомогательные методы поиска элементов формы (*FindTextBox*, *FindButton*, *FindLabel*) и тесты валидации ввода — ввод букв вместо числа и пустого поля.


```

    }

    [TestMethod]
    Ссылка: 0
    public void Test20_OperationSelected()
    {
        MainForm form = new MainForm();
        Button btn = FindButton(form, "btnMul");
        form.OpClick(btn, EventArgs.Empty);

        ICalculatorUI ui = form;
        Assert.AreEqual("x", ui.ReadOperation());
    }

    [TestMethod]
    Ссылка: 0
    public void Test21_CalcButtonShowsResult()
    {
        MainForm form = new MainForm();
        FindTextBox(form, "re1Box").Text = "2";
        FindTextBox(form, "im1Box").Text = "3";
        FindTextBox(form, "re2Box").Text = "1";
        FindTextBox(form, "im2Box").Text = "4";

        form.OpClick(FindButton(form, "btnAdd"), EventArgs.Empty);
        form.CalcClick(form, EventArgs.Empty);

        string text = FindLabel(form, "resultLabel").Text;
        StringAssert.Contains(text, "Результат");
        StringAssert.Contains(text, "3");
        StringAssert.Contains(text, "7");
    }

    [TestMethod]
    Ссылка: 0
    public void Test22_DisplayCompareResult()
    {
        MainForm form = new MainForm();
        FindTextBox(form, "re1Box").Text = "2";
        FindTextBox(form, "im1Box").Text = "3";
        FindTextBox(form, "re2Box").Text = "2";
        FindTextBox(form, "im2Box").Text = "3";

        form.OpClick(FindButton(form, "btnEq"), EventArgs.Empty);
        form.CalcClick(form, EventArgs.Empty);

        string text = FindLabel(form, "resultLabel").Text;
        StringAssert.Contains(text, "true");
    }
}

```

Рисунок-6 Тесты выбора операции и вывода результата

Тесты интерфейсной части *MainForm*: проверка выбора операции через кнопку, отображение результата вычисления на метке *resultLabel*, а также отображение результата операции сравнения двух комплексных чисел.

Результаты прогона тестов

Тесты запускались командой `dotnet test`. Прошли все 28 тестов:

Пройден! : не пройдено 0, пройдено 28, пропущено 0, всего 28

Из них 23 теста проверяют класс *ComplexNum* (арифметика, сравнение, модуль, форматирование) и 5 тестов — интерфейсную часть *MainForm* (валидация ввода, выбор операции, отображение результата).

Заключение

В ходе выполнения лабораторной работы №5 на основании сформированных ранее тестовых сценариев были реализованы модульные тесты для калькулятора комплексных чисел. Тесты покрывают все операции класса *ComplexNum* (сложение, вычитание, умножение, деление, сравнение на равенство, сравнение по модулю), а также интерфейсную часть программы — валидацию пользовательского ввода, выбор операции и вывод результата в форме *MainForm*. Все 28 тестов прошли успешно, что подтверждает корректность реализованного функционала.